



Trabajar con PlatformIO desde OpenCode

→ ESP32-S3 • PlatformIO Core 6.2 • Sin salir de la terminal

¿Por qué es posible?

PlatformIO es, en su núcleo, una **línea de comandos** (pio). La extensión de VSCode es solo una interfaz gráfica sobre esa misma CLI. Por lo tanto, cualquier terminal —incluida la de OpenCode— puede ejecutar `pio run`, `pio device monitor`, subir firmware, gestionar bibliotecas y configurar `platformio.ini` con el mismo resultado.

1 Instalación en este workspace

1. Instalar el Core en el entorno conda `elect_env` (ya hecho):

```
1 python3 -m pip install platformio
```

2. Verificar que la shell de OpenCode lo resuelve (el plugin `conda-env.js` antepone `elect_env/bin` al PATH de cada shell):

```
1 pio --version
2 # PlatformIO Core, version 6.2.0
```

3. Opcional: registrar el servidor MCP `platformio-mcp` en `opencode.json` para que OpenCode exponga herramientas nativas de PlatformIO (board discovery, build, flash, monitor, diagnóstico):

```
1 "mcp": {
2   "platformio": {
3     "type": "local",
4     "command": ["npx", "-y", "platformio-mcp"],
5     "enabled": true,
6     "environment": {
7       "PATH": "/home/cero/anaconda3/envs/elect_env/bin:/home/cero/.
          nvm/versions/node/v22.23.1/bin:/usr/bin:/bin"
8     }
9   }
10 }
```

Requiere reiniciar OpenCode para que el servidor se cargue.

2 Flujo básico de trabajo

1. Crear/inicializar un proyecto:

```
1 pio project init --board esp32-s3-devkitc-1 --project-option "
  framework=arduino"
```

2. Compilar (equivalente al botón Build de VSCode):

```
1 pio run
```

3. Subir firmware (equivalente a Upload):

```
1 pio run --target upload
```

4. Monitor serie integrado:

```
1 pio device monitor
```

5. Ver puertos disponibles:

```
1 pio device list
```

3 Regla: comandos deterministas, no decisiones en el chat

En ELECTRONICA la lógica repetible vive en scripts deterministas, no improvisada en el chat. Para flujos PlatformIO reutilizables se sigue la arquitectura de 3 capas: directiva YAML en `directives/`, orquestador en la raíz (o servidor MCP), y ejecución en `execution/` con `pio` vía subprocess. Un comando puntual (compilar, subir) puede correr directamente desde la shell; un flujo repetible debe quedar documentado como script.

4 Vista rápida: VSCode vs OpenCode

Comparativa

Operación	VSCode (extensión)	OpenCode (CLI/MCP)
Compilar	Botón Build	<code>pio run</code> o herramienta MCP
Subir firmware	Botón Upload	<code>pio run -target upload</code>
Monitor serie	Panel integrado	<code>pio device monitor</code>
Libs	UI gestor de librerías	<code>pio pkg install <lib></code>
Diagnóstico	PlatformIO Home	<code>MCP agent_build_diagnose</code>
Edición de código	Editor + IntelliSense	OpenCode + tools de edición

Limitación honesta: VSCode aporta IntelliSense, resaltado de sintaxis y un monitor serie gráfico que OpenCode no replica. Para edición idéntica, nada impide usar VSCode como editor y ejecutar `pio` desde su terminal integrada; con OpenCode ganas que el agente IA compile, suba y diagnostique por ti sin cambiar de herramienta.

Verificado en este workspace (2026-09-23): `pio run` de un blink para `esp32-s3-devkitc-1` finalizó con éxito en 79 s (RAM 5.7 %, Flash 7.5 %) usando PlatformIO Core 6.2.0 sobre `elect_env`.